



中国计算机学会
China Computer Federation



范围修改查询问题

——清华大学 李欣隆



中国计算机学会
China Computer Federation



自我介绍

- NOI2020
- WC2022
- CTS2021, CTS2022
- CTT2020, CTT2021
- THUWC2020, THUSC2022
- THUPC2019, THUPC2021, THUPC2022
- 参与命题



中国计算机学会
China Computer Federation



抽象数据结构

- 不考虑具体维护什么信息，考虑在维护的信息满足某些特定条件下的通用问题形式。





抽象数据结构

- 原始问题:
- 维护一个序列, 支持区间加, 询问区间max
- 维护一个序列, 支持区间加, 询问区间min
- 维护一个序列, 支持区间对一个数取max, 询问区间max
- 维护一个序列, 支持区间修改为一个给定的数, 询问区间和
- 维护一个序列, 支持区间修改为一个给定的数, 询问区间最大子段和



抽象数据结构

- 抽象后的问题：
- 维护一个序列，可以用某种数据表示序列上一个区间的信息，可以用某种数据表示序列区间的一次修改，满足的性质：
 - 1. 序列上两个区间 $[a,b]$, $[b+1,c]$ 的信息可以 $O(1)$ 合并出 $[a,c]$ 的信息。
 - 2. 对一个区间进行的相邻两次修改操作可以 $O(1)$ 合并为一次修改操作。
 - 3. 如果知道了一个区间的信息，可以 $O(1)$ 计算出这个区间经过一次修改后的信息。



抽象数据结构

- 维护一个序列，支持区间加，询问区间max
- 用什么数据表示：用一个int表示每个区间的max，用一个int表示加的量
- 1. 假设 $[a,b]$ 的最大值为 x ， $[b+1,c]$ 的最大值为 y ，则 $[a,c]$ 的最大值为 $\max(x,y)$ 。
- 2. 假设 $[a,b]$ 加了 x ，之后 $[a,b]$ 加了 y ，等价于 $[a,b]$ 加了 $(x+y)$ 。
- 3. 假设 $[a,b]$ 的最大值为 x ，之后 $[a,b]$ 加了 y ，则 $[a,b]$ 的最大值为 $x+y$ 。



抽象数据结构

- 维护一个序列，支持区间修改为一个给定的数，询问区间最大子段和
- 用什么数据表示：用四个int表示每个区间的 (最大前缀和，最大后缀和，最大子段和，和)，用一个int表示修改为什么数
- 1. 假设 $[a,b]$ 的信息为 $(x1,y1,z1,w1)$ ， $[b+1,c]$ 的信息为 $(x2,y2,z2,w2)$ ，则 $[a,c]$ 的最大值为 $(\max(x1,x2+w1), \max(y1+w2,y2), \max(z1,z2,y1+x2), w1+w2)$ 。
- 2. 假设 $[a,b]$ 修改为 x ，之后 $[a,b]$ 修改为 y ，等价于 $[a,b]$ 修改为 y 。
- 3. 假设 $[a,b]$ 的信息为 (x,y,z,w) ，之后 $[a,b]$ 修改为 c ，则 $[a,b]$ 的信息为 $\max((0,0,0,0), (c*(b-a+1), c*(b-a+1), c*(b-a+1), c*(b-a+1)))$ 。



中国计算机学会
China Computer Federation



半群

- 给定一个集合 D ，以及 D 上的一个二元运算 $*$ ： $D \times D \rightarrow D$
- 满足结合律： $(a*b)*c=a*(b*c)$





中国计算机学会
China Computer Federation



么半群

- 给定一个集合 D ，以及 D 上的一个二元运算 $*$ ： $D \times D \rightarrow D$
- 满足结合律： $(a*b)*c=a*(b*c)$
- 有么元： $a*\varepsilon=\varepsilon*a=a$



中国计算机学会
China Computer Federation



交换半群

- 给定一个集合 D ，以及 D 上的一个二元运算 $*$ ： $D \times D \rightarrow D$
- 满足结合律： $(a*b)*c=a*(b*c)$
- 满足交换律： $a*b=b*a$



中国计算机学会
China Computer Federation



抽象信息

- 前面的两个例子中维护的信息可以看做是半群信息。
- 可以发现有两个半群，一个表示区间的信息，一个表示修改的信息，即标记。



中国计算机学会
China Computer Federation



更多的范围

- 修改和询问的范围不一定是序列的区间
- 序列区间，树简单路径，二维平面矩形，高维正交范围，半平面范围，圆范围……



对范围进行抽象

- 一个范围可以看做是初始点集的一个子集。
- 初始有 n 个点，可以用某种数据表示一个范围信息，可以用某种数据表示范围的一次修改，满足的性质：
 1. 两个范围 S_1, S_2 的信息可以 $O(1)$ 合并出 $S_1 + S_2$ 的信息。
 2. 对一个范围进行的相邻两次修改操作可以 $O(1)$ 合并为一次修改操作。
 3. 如果知道了一个范围的信息，可以 $O(1)$ 计算出这个范围经过一次修改后的信息。



范围修改查询问题的抽象数据结构模型

- OI里常见的描述方法：
- 给一些点，每个点有个初始权值。
- 每次修改给一个范围以及一个修改的值，表示把范围内的点的权值与修改权值做一个二元运算。
- 每次查询给一个范围，表示把范围内的点的权值求和。
- 这里有两种数据类型和两种二元运算，数据类型为点的权值与修改的权值，二元运算为修改对点权的影响，以及点权的合并。
- 要求修改可合并（标记的合并）。



范围修改查询问题的抽象数据结构模型

给一个初始集合 I_0 , 一个有限集合 $I \subset I_0$, 询问集合 $Q \subset 2^{I_0}$ 满足 $I_0 \in Q$, 交换半群 $(\mathcal{D}, +)$, 半群 $(\mathcal{M}, *)$, 二元运算符 $* : \mathcal{D} \times \mathcal{M} \rightarrow \mathcal{D}$.

结合律: $\forall a \in \mathcal{D}, b, c \in \mathcal{M}, (a * b) * c = a * (b * c)$.

交换律: $\forall a, b \in \mathcal{D}, c \in \mathcal{M}, (a + b) * c = a * c + b * c$.

初始权值 $d_0 : I \rightarrow \mathcal{D}$.

按顺序执行 m 个操作, 每个操作是一个修改或者查询操作。

第 t 个操作定义为:

1. 范围修改: 给出 $q_t \in Q$ 和 $f_t \in \mathcal{M}$, 定义 $d_t(x) = d_{t-1}(x) * f_t$ 如果 $x \in q_t$, 否则 $d_t(x) = d_{t-1}(x)$.
2. 范围查询: 给出 $q_t \in Q$, 定义 $d_t(x) = d_{t-1}(x)$ 对所有 $x \in I$, 答案为 $\sum_{x \in q_t} d_t(x)$.

所有操作以及初始权重均离线给出。

我们用 $n = |I|$ 表示初始权重的个数, 用 m 表示范围修改查询问题的操作数。



范围修改查询问题的抽象数据结构模型

- “双半群”模型。
- 初始给定一个大小为 n 的点集，有 m 次操作。
- 每个点的信息为交换半群 D 中的元素，修改为半群 M 中的元素。
- D 上的一个二元运算 $+$: $D + D \rightarrow D$
- M 上的一个二元运算 $*$: $M \times M \rightarrow M$
- 修改对信息的影响是二元运算 $*$: $D \times M \rightarrow D$
- $*$ 对 $+$ 有分配律。
- 通俗来讲， D 表示一个范围的信息合并后的结果， M 表示修改打的标记。之后看到点集默认为初始给定的这个点集。



通用程度的比较

- 1. 给一个序列，支持区间加，询问区间 $\max$ 。
- 2. 给一个序列，每个位置初始为 D 中的元素，支持区间乘一个 M 中的元素，询问区间和， D, M 满足上述“双半群”模型的性质。
- 3. 初始给一些点，每个点上有一个 D 中的元素，支持范围乘一个 M 中的元素，询问范围积， D, M 满足上述“双半群”模型的性质，根据具体问题，范围可以有很多种，比如区间，矩形，树简单路径，半平面，多边形，圆，半空间等。
- 1是一个具体问题，2是一类同样范围但代数结构不同的问题，3是一类不同范围不同代数结构的问题，要做3的话我们的算法要能在不同范围下都适用。



M-表达式

$\mathcal{M}$ -expr 归纳定义如下:

$\mathcal{M}$ 中的一个元素是一个 $\mathcal{M}$ -expr.

如果 a, b 都是 $\mathcal{M}$ -expr, 则 $(a * b)$ 是一个 $\mathcal{M}$ -expr.

一个 $\mathcal{M}$ -expr 可以通过使用上述规则有限次被构造出。

- 构造M-表达式时, 初始情况为一个修改操作的权值, 每递归一次会在后面乘上一个修改操作的权值。
- M-表达式是一连串修改操作对应的表达式。



D-表达式

$\mathcal{D}$ -expr 归纳定义如下:

$\mathcal{D}$ 中的一个元素是一个 $\mathcal{D}$ -expr.

如果 a, b 都是 $\mathcal{D}$ -expr, 则 $(a + b)$ 是一个 $\mathcal{D}$ -expr.

如果 a 是一个 $\mathcal{D}$ -expr, b 是一个 $\mathcal{M}$ -expr, 则 $(a * b)$ 是一个 $\mathcal{D}$ -expr.

一个 $\mathcal{D}$ -expr 可以通过使用上述规则有限次被构造出。

- D-表达式是一个范围的信息的和。
- 构造D-表达式时, 初始情况为一个点的权值, 每递归一次要么合并两个范围的权值, 要么乘一个修改操作的权值, 即进行修改。



中国计算机学会
China Computer Federation



子表达式

子表达式定义为上述规则的反, 传递闭包, 即:

对于一个表达式 a , a 是 a 的子表达式。

对于表达式 a, b, c , 如果 a 是 b 的子表达式, b 是 c 的子表达式, 则 a 是 c 的子表达式。

- 子表达式即表达式树的每个子树对应的表达式。





时间复杂度

- 这里考虑的时间复杂度是与具体信息无关的。
- 比如区间加区间 max 这样的问题，我们已知的做法是线段树，但我们怎么证明必须用线段树或者同复杂度的做法才能解决这个问题？因为加，max 这两个操作可能有一些特殊的性质，所以我们很难证明不存在更优的做法。
- 但是在我们这里的计算模型下，复杂度下界是可以被证明的，因为这个双半群模型能用到的性质比具体问题更少。



标准型

- 一个表达式与具体 D, M 的结构以及每个参数具体的取值无关。
- 当给出具体的代数结构 D, M 以及每个输入的权值具体是代数结构中的哪个元素时，我们可以计算出一个表达式的值。
- 由于我们不关心在一组特定的数据下算法的时间复杂度，所以可以认为所有输入的元素（即初始权值以及每次修改的权值）均是变量。
- 因为 D -表达式 可以表示一个范围内的和，所以每次询问的答案即为对应的 D -表达式。



中国计算机学会
China Computer Federation



最优离线范围修改查询算法

- 我们提出了一个最优的离线范围修改查询算法（当然这里的最优是需要满足一些条件的，不过这个算法足够通用）。
- 这个算法是可以被简单实现，并且常数较小的。





离线的意义

- 给定一个长为 n 的序列，有 m 次操作，每次操作是区间修改或查询， $n \gg m$ 。
- 在线做法是对序列建立线段树，时间复杂度 $O(n + m \log n)$ ，可以证明这个复杂度是确界，即不存在时间复杂度更低的算法。
- 离线做法是离散化，将这 m 次操作的端点提出，序列中每段都被进行了相同的操作（即对每次修改操作，要么包含了一段中所有位置，要么不包含一段中的任何位置），所以可以预先合并每段的信息，把序列长度变成 $O(m)$ ，再对序列建立线段树，时间复杂度 $O(n + m \log m)$ 。
- 离线由于知道每次操作的范围，可以优化操作次数。

分治

- 由于修改操作预先给定，可以设计一个在修改操作的序列上分治的算法。





等价类

假设操作 $q_l, q_{l+1} \dots q_r$ 表示编号为 $[l, r]$ 的操作，定义等价关系

$R_{l,r}(x, y) = \forall i \in [l, r], x \in q_i \leftrightarrow y \in q_i$ 在 I 上成立， q_i 表示第 i 次操作的范围。

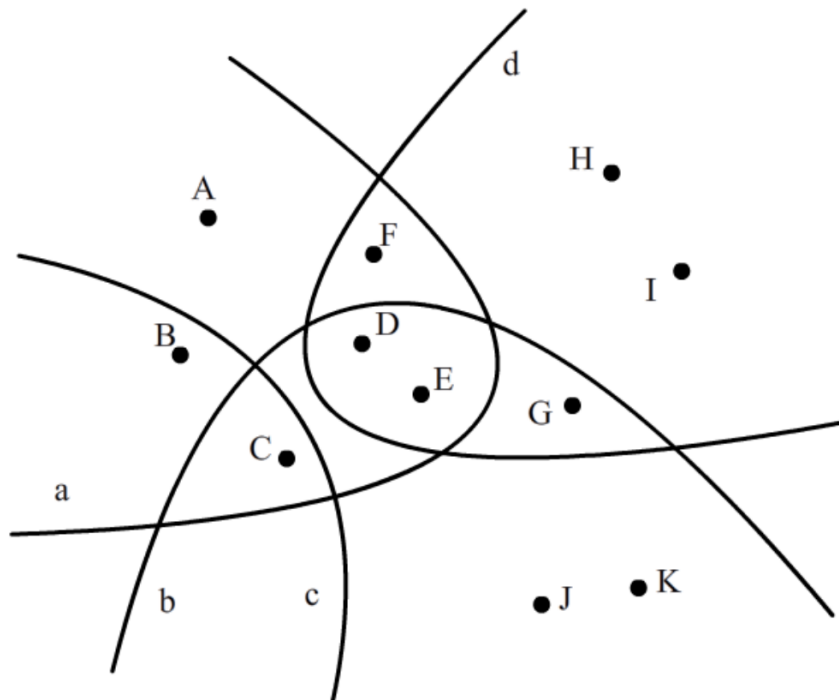
如果 $[l', r'] \subset [l, r]$ ，则 $\forall x, y \in I, R_{l,r}(x, y) \rightarrow R_{l',r'}(x, y)$ ，即 $R_{l,r}$ 将 I 划分为比 $R_{l',r'}$ 更细的部分。我们把这个关系写作 $R_{l,r} \subseteq R_{l',r'}$ 。

- 这里的意思是两个点 x, y 在操作区间 $[l, r]$ 中在同一个等价类里，等价于对任何编号在 $[l, r]$ 中的操作范围， x, y 这两个点要么被这个操作范围同时包含，要么同时不被这个操作范围包含。
- 划分为更细的部分这里只需要理解为，如果没有操作，那所有点都属于同一个等价类，如果有很多操作，那基本上每个等价类里面只有几个点。如果慢慢加操作，则越加划分会越细，最粗的划分就是只有一个等价类，最细的划分是有 n 个等价类。



中国计算机学会
China Computer Federation

等价类



■ **Figure 1** In this figure, $I_0 = \mathbb{R}^2$, $I = \{A, B, \dots, K\}$, each element in Q is one side of a curve, $q_1, q_2, q_3, q_4 = a, b, c, d$ are the boundary of ranges of operations. These four ranges partition I into 8 equivalence class $\{A\}, \{B\}, \{C\}, \{D, E\}, \{F\}, \{G\}, \{H, I\}, \{J, K\}$.

$I_0 = \mathbb{R}^2, I = \{A, B, \dots, K\}$, 每个 Q 中的元素是双曲线一侧的点集, $q_1, q_2, q_3, q_4 = a, b, c, d$ 是操作的范围, 这 4 个范围将 I 划分为 $\{A\}, \{B\}, \{C\}, \{D, E\}, \{F\}, \{G\}, \{H, I\}, \{J, K\}$ 这 8 个等价类。



中国计算机学会
China Computer Federation



恒等修改

为了方便我们引入幺元 $\epsilon_{\mathcal{M}}$, 令 $\mathcal{M}' = \mathcal{M} \cup \{\epsilon_{\mathcal{M}}\}$ 为幺半群, 满足定义:

$$\forall x \in \mathcal{D}, x * \epsilon_{\mathcal{M}} = x$$

$$\forall x \in \mathcal{M}, x * \epsilon_{\mathcal{M}} = \epsilon_{\mathcal{M}} * x = x$$





中国计算机学会
China Computer Federation



动态有根森林

定义一个等价关系 R_0 满足每个初始给定的点在这个等价关系下都属于不同的等价类。为了支持修改和查询，我们维护一个动态有根森林，每个叶子为初始给定的一个点。两个叶子节点在同一个子树中，当且仅当这两个叶子对应的点在当前等价关系下属于同一个等价类。在初始情况下森林中存在 $|I| = n$ 棵有根树，每棵树只有一个点，这个有根森林对应于等价关系 R_0 。





等价关系间的移动

假设当前森林对应于 R_1 , 给出 R_2 满足 $R_1 \subseteq R_2$, 我们可以从目前 R_1 的状态移动到 R_2 的状态, 这里具体来说指的是通过合并一些有根树, 使得我们有根森林的状态变为等价于 R_2 . 合并两棵以 x_1, x_2 为根的树的方法是新建一个点 z 做为 x_1, x_2 的父亲结点。同样的, 如果我们的有根森林状态对应于 R_2 , 我们也可以通过撤销一些合并操作, 将其改变回 R_1 .

- 这里注意两个等价关系间**移动**的具体定义, 之后这个概念会大量使用。



额外的域（这里的域就是成员变量的意思）

为了实现范围修改操作，我们使用众所周知的懒惰标记的方法。

在每个结点 x 上我们维护两个额外的域（你们理解为多开两个int存东西就行）来存储 $x.d_0$ 做为懒惰标记，以及 $x.d \in \mathcal{D}$ 表示子树的和（这里的和前面讲的半群代数结构上的）。

对于每个结点 x 与一条自顶向下的路径 $(x_1, \dots, x_k)$ ，其中 x_1 是根，且 $x_k = x$ ，定义 $x.d_0 = x.d * x_{k-1}.u * \dots * x_2.u * x_1.u$ ，表示经过根到结点 x 的标记作用后 x 的子树和。我们要维护子树的和，于是：

$$x_1.d_0 = x_1.d$$

$$x.d_0 = \sum_{y \text{ is child of } x} y.d_0$$

所有动态有根森林的操作都需要保证上述 $x.d_0$ 的性质，通过维护上述性质可以保证答案的计算是正确的。



有根森林的加点删点

我们可以新建一个结点 z 做为根 $(x_1, x_2, \dots, x_k)$ 的父亲, 从而合并这些有根树为一棵新的有根树, 这里 z 的额外域为 $z.u = \epsilon_M, z.d = \sum_{i=1}^k x_i.d$.

我们可以将儿子 $(x_1, \dots, x_k)$ 的父亲结点 z 删除, 从而将一棵有根树分裂为 k 棵有根树, 这里我们需要下放 z 的标记, x_k 的额外域从 $(x_k.d, x_k.u)$ 变为 $(x_k.d * z.u, x_k.u * z.u)$.

如果有一棵根为 x 的树, 其包含的所有叶子结点恰好对应于一次修改 u 包含的所有点, 我们可以对 x 打标记 $(x.d, x.u) \leftarrow (x.d * u, x.u * u)$. 如果这是一次对同样范围的询问操作, 则 $x.d$ 是这次询问的答案。



类型

- 维护的动态树森林中每棵树对应一个点集。
- 每次修改或者询问也是对一个点集进行操作的。
- 每棵树的根节点都存了这棵树对应的等价类中点信息的和，以及对这棵树对应的等价类中的所有点进行的操作的标记。
- 可以通过加一个父亲来合并多棵有根树，这对应于我们合并了多个点集，并将信息相加，即信息上传。
- 可以通过删除一个父亲来拆散多棵有根树，这对应于我们拆分了多个点集，并将之前对这些点集的修改操作的影响施加，即标记下传。

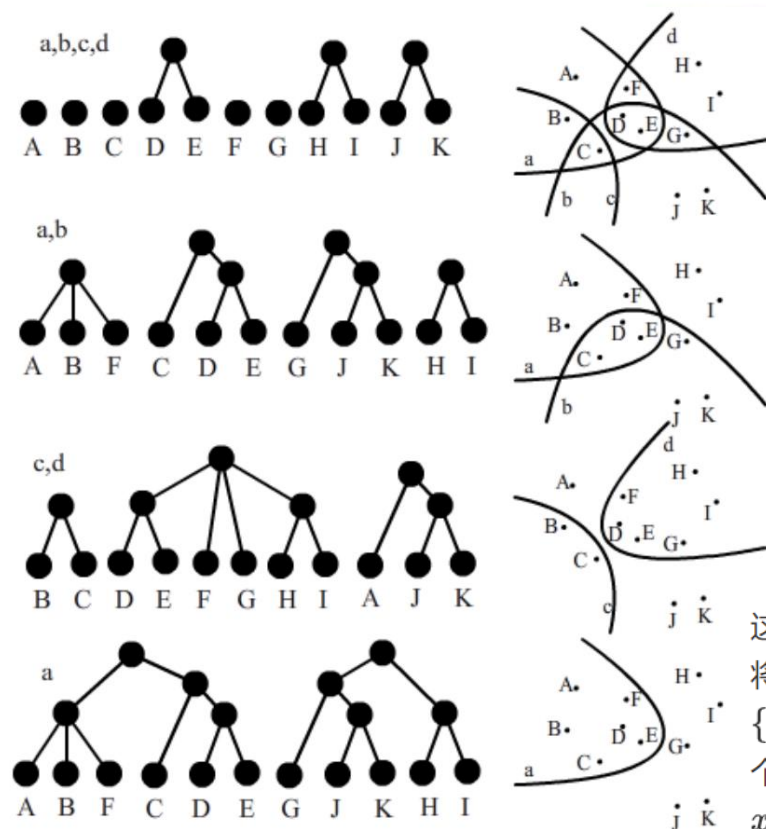


分治

我们的算法将操作序列分为一些阶段。假设一个阶段包含了操作 $q_l, q_{l+1}, \dots, q_r$ ，假设对所有 $x \in I$ 都已知 $d_{l-1}(x)$ ，我们先将等价关系从 R_0 变为 $R_{l,r}$ ，然后对 $[l, r]$ 使用我们的分治算法，然后由 $R_{l,r}$ 撤销回 R_0 从而得到对所有 $x \in I$ 的 $d_r(x)$ ，这样即计算出了下一个阶段中每个点的初始值。

当我们在进行 $q_l, q_{l+1}, \dots, q_r$ 上的分治时我们将等价关系从 $R_{l,r}$ 移动到 $R_{l,m}$ ，之后递归地在 $q_l, \dots, q_m$ 上跑我们的分治算法。当我们完成在 $q_l, \dots, q_m$ 上的计算后，我们撤销一些合并操作从而将 $R_{l,m}$ 撤销回 $R_{l,r}$ ，然后移动到 $R_{m+1,r}$ ，对 $q_{m+1}, \dots, q_r$ 跑我们的分治算法，然后移动回 $R_{l,r}$ 。当递归到 $l = r$ 时，意味着只有一个操作，也就是一个范围，所以只有 2 个等价类。如果是修改操作，则在对应 q_l 的范围的有根树的根上打上对应的修改标记，如果是询问操作，则根的 d 是这次询问的答案。

例子



这张图演示了当等价关系为 $R_{1,4}, R_{1,2}, R_{3,4}, R_{1,1}$ 时有根森林的结构。我们从 $R_{1,4}$ 移动到 $R_{1,2}$ 时，将等价类 $\{A\}, \{B\}, \{F\}$ 合并为 $\{A, B, F\}$; $\{C\}, \{D, E\}$ 合并为 $\{C, D, E\}$; $\{G\}, \{J, K\}$ 合并为 $\{G, J, K\}$ 。我们从 $R_{1,2}$ 移动到 $R_{1,4}$ 时，我们可以撤销上述合并。当递归到 $l = r$ 时，最多只有 2 个等价类，对于 $R_{1,1}$ ，设 x 是对应于等价类 $\{A, B, C, D, E, F\}$ 的有根树的根，我们打上修改标记 $x.u = x.u * f_1$ 来进行这次修改操作。

Figure 2 This figure corresponds to the structure of the rooted forests and equivalence classes when our equivalence relation is $R_{1,4}, R_{1,2}, R_{3,4}, R_{1,1}$. When we move from $R_{1,4}$ to $R_{1,2}$, we merge equivalence classes $\{A\}, \{B\}, \{F\}$ into $\{A, B, F\}$; $\{C\}, \{D, E\}$ into $\{C, D, E\}$; $\{G\}, \{J, K\}$ into $\{G, J, K\}$, and when we move from $R_{1,2}$ back to $R_{1,4}$ we can undo these merges. For the corner case that $l = r$, there are at most 2 equivalence classes, for $R_{1,1}$, let x be the root of tree corresponding to equivalence class $\{A, B, C, D, E, F\}$ we can update $x.u = x.u * f_1$.



区域数

定义离散函数 F , $F(k)$ 是 the maximum combinatorial complexity of an arrangement including k operations, 也就是 k 个操作最多将 I_0 划分为多少个等价类。 k 个操作能将 I 划分出的等价类数不超过 $\min(F(k), |I|)$.

显然 $F(k)$ 单调不下降, 于是我们可以定义离散函数 G , $G(n)$ 满足 $F(G(n)) \leq n$, $F(G(n) + 1) > n$. 这里 $G(n)$ 的意义即给定范围的情况下, 需要多少个操作才能将点集划分为最细的等价关系, 即 n 个等价类。

- 这里的意思是少数操作只能将初始点集划分为少量等价类, 具体 k 个操作能划分出多少个区域需要根据具体问题计算, 但这里的计算一般比较简单。



分块

定义离散函数 F , $F(k)$ 是 the maximum combinatorial complexity of an arrangement including k operations, 也就是 k 个操作最多将 I_0 划分为多少个等价类。 k 个操作能将 I 划分出的等价类数不超过 $\min(F(k), |I|)$.

显然 $F(k)$ 单调不下降, 于是我们可以定义离散函数 G , $G(n)$ 满足 $F(G(n)) \leq n$, $F(G(n) + 1) > n$. 这里 $G(n)$ 的意义即给定范围的情况下, 需要多少个操作才能将点集划分为最细的等价关系, 即 n 个等价类。

我们将给定的 m 个操作分为一些阶段, 每个阶段包含连续的 $G(n)$ 个操作, 在每个阶段中我们使用上述的分治算法。代数结构上的操作次数为

$$T = O(n) + \frac{m}{G(n)} T_1(G(n)), \quad T_1(m) = 2T_1(m/2) + O(F(m)).$$



中国计算机学会
China Computer Federation



常见范围

对于区间修改查询操作，树上简单路径修改查询操作，我们有 $F(k) = \Theta(k)$ ，在这种情况下 $T(m) = O(n + m \log \min(n, m))$;

对于修改查询范围为半平面，正交矩形，多边形，圆，椭圆，双曲线等情况， $F(k) = \Theta(k^2)$ ，在这种情况下 $T(m) = O(n + m\sqrt{n})$;

对于修改查询范围为 c 维空间范围，如 c 维半空间，单纯性，球，正交范围，多面体，椭球，有 $F(k) = \Theta(k^c)$ ， $c > 1$ ，在这种情况下 $T(m) = O(n + mn^{1-1/c})$. 我们同时可以证明这种情况下代数结构上的运算次数下界为 $\Omega(n + mn^{1-1/c})$ ，这说明我们的算法在这种情况下是最优的。

对于我们的算法，代数结构上的运算次数只和 n, m, I_0, Q 以及 F 函数有关，与修改查询的具体范围以及具体代数结构无关。代数结构上的运算次数可以被看做是一个衡量一个范围修改查询问题难度的通用的指标。



中国计算机学会
China Computer Federation



代数结构运算次数

- 可以看做是一道交互题。
- 给了你交互函数接口，让你可以操作 D, M 中的元素，你每进行一次 D, M 中的元素的运算，交互库就会记录你进行的代数结构运算次数+1。
- 在讨论代数结构运算次数时，我们只关心交互库记录的那个值，不关心我们具体的时间复杂度，也就是说找出每次合并的树和撤销的树可以暴力实现。



[CTS2021] 测测你的半平面修改查询

这是一道交互题。

给定平面上的 n 个点，第 $i + 1$ 个点为 (x_i, y_i) ，初始值为 d_i ， $0 \leq i < n$ ；

你需要按顺序进行 m 次操作，所有操作在一开始已知；

第 $j + 1$ 次操作给出三个数 a_j, b_j, c_j ，以及 o_j ，表示半平面 $a_j x + b_j y < c_j$ ， $0 \leq j < m$ ；

你需要按顺序执行：

首先令 $s_j = \epsilon_D$ ，然后对每个满足 $a_j x_i + b_j y_i < c_j$ 的 i ，先将 s_j 修改为 $s_j + d_i$ ，后将 d_i 修改为 $o_j \cdot d_i$ ；

完成这些修改后，就得到了这次操作的答案 s_j ；

其中 d_i 属于抽象的数据类型 D ， o_j 属于抽象的数据类型 O ；

D 上定义了抽象的运算 $+$ ： $D \times D \rightarrow D$ ；

D, O 上定义了抽象的运算 $\cdot$ ： $O \times D \rightarrow D$ ；

O 上定义了抽象的运算 $\cdot$ ： $O \times O \rightarrow O$ ；

ϵ_D 是 D 中的一个特殊的元素，称为单位元；

ϵ_O 是 O 中的一个特殊的元素，称为单位元；

这些操作满足性质：

对任意 $a, b, c \in D$ ，有 $a + b = b + a$ ， $(a + b) + c = a + (b + c)$ ， $a + \epsilon_D = \epsilon_D + a = a$ ；

对任意 $u, v, w \in O$ ，有 $(u \cdot v) \cdot w = u \cdot (v \cdot w)$ ， $u \cdot \epsilon_O = \epsilon_O \cdot u = u$ ；

对任意 $u, v \in O$ ， $a, b \in D$ ，有 $(u \cdot v) \cdot a = u \cdot (v \cdot a)$ ， $u \cdot (a + b) = (u \cdot a) + (u \cdot b)$ ， $\epsilon_O \cdot a = a$ ， $u \cdot \epsilon_D = \epsilon_D$ ；

执行每次 $+$ 或 $\cdot$ 运算有一定的代价，具体地，在计算 $a + b$ 或 $a \cdot b$ 时如果 a, b 都不是 ϵ_D 或 ϵ_O ，则代价为1，否则代价为0。你需要保证每个答案正确，且总代价不能超过当前子任务的**代价上限**。



子任务	n	m	代价上限	分值	依赖
1	2×10^4	10^3	4×10^7	5	
2	2×10^4	10^3	8×10^5	15	1
3	2×10^5	10^2	4×10^7	5	
4	2×10^5	10^2	1.25×10^6	15	3
5	2^{16}	10^4	8×10^7	20	
6	2×10^5	10^4	2.5×10^7	10	
7	2×10^5	10^4	1.25×10^8	15	
8	2×10^5	10^4	2.5×10^7	15	1~7



找区域

对于半平面修改查询的问题, $F(k) = \Theta(k^2)$, 于是我们将时间分为一些大小为 $\Theta(\sqrt{n})$ 大小的阶段, 每个阶段有 $O(n)$ 个区域。我们使用平面图点定位的算法, 从 $q_l, \dots, q_r$ 移动到 $q_l, \dots, q_m$ 和 $q_{m+1}, \dots, q_r$ 时的时间复杂度为 $\tilde{O}((r-l+1)^2 + n)$, 总时间复杂度为 $\tilde{O}(n + m\sqrt{n})$.

- 这个算法可以在同复杂度完成圆, 双曲线, 椭圆等范围修改查询。





一种哈希方法

有一个通用的基于哈希的方法，我们从 R_0 移动到 $R_{l,r}$ 时可以对 $q_l, \dots, q_r$ 这些范围进行范围加，每个范围随机一个在 $[0, 2^w)$ 中的整数。当范围修改结束后，我们询问 I 中每个点的值。之后每个 $R_{l,r}$ 中的等价类都应该有同样的哈希值，而且不同的等价类应该大概率有不同的哈希值。我们对每个 $R_{l,r}$ 中的等价类找一个代表元，将 I 变为一个更小的点集。然后我们进行从 $R_{l,r}$ 到 $R_{l,m}$ 与 $R_{m+1,r}$ 的移动，并且递归使用我们的分治算法。

- 问题转换为进行一些范围加之后输出每个点的值，这里是一个新的但是更简单的抽象数据结构问题。



中国计算机学会
China Computer Federation



半在线性质

- 我们的算法可以一定程度上半在线，即第 i 次修改的值依赖于前 $i-1$ 次询问的值。
- 但是需要至少每次知道一个阶段（即时间块）内所有操作的范围。



复杂度下界

- 我们证明我们的算法在代数结构运算次数上是渐进最优的。
- 因为范围可能非常奇怪，所以我们只关心代数结构运算次数，其余部分不关心。
- 约定范围可以用常数个 `int` 储存，以及范围之间的常见运算均为 $O(1)$ 。
- 这里对范围进行限制是合理的，不然解决了一个过于通用的问题，可以构造出比如“当一个程序停机时对区间 $[l, r]$ 加 A ”这样的离谱形式。当你做的问题过于通用时很难得出什么非平凡的结论，所以这里需要限制通用性。



列表

List 列表

一个列表 $a = (a_1, a_2, \dots, a_n)$ 有长度 $\text{len}(a) = n$, 和 n 个元素 $a_1, a_2, \dots, a_n$.

两个列表 a, b 相等当且仅当 $\text{len}(a) = \text{len}(b)$, 且 $\forall i \in \mathbb{N} \cap [1, \text{len}(a)], a_i = b_i$.

一个空列表写作 $()$.

列表连接:

可以将列表 a 与 b 连接, 表示为 $a + b$.

$$\text{len}(a + b) = \text{len}(a) + \text{len}(b).$$

$$\forall i \in \mathbb{N} \cap [1, \text{len}(a)], (a + b)_i = a_i.$$

$$\forall i \in \mathbb{N} \cap [1, \text{len}(b)], (a + b)_{\text{len}(a)+i} = b_i.$$

列表元素计数:

$$\text{count}((a_1, \dots, a_n), x) = \sum_{i=1}^n [a_i = x], \text{ 其中 } [\text{cond}] = 1 \text{ 如果 } \text{cond} \text{ 是真, 否则 } [\text{cond}] = 0.$$



中国计算机学会
China Computer Federation



可重集合

Multiset 可重集合

一个可重集合是一个列表，但是定义相等的方法和列表不同。

我们使用 $\{a_1, \dots, a_n\}$ 来表示列表 $(a_1, \dots, a_n)$ 对应的可重集合。

两个 a, b 相等当且仅当 $\forall x, \text{count}(a, x) = \text{count}(b, x)$.

两个可重集合的不去重并定义为将这两个可重集合的列表连接之后将这个列表转换为可重集合。

映射操作：

如果 $a = \{a_1, \dots, a_n\}$ 是一个可重集合，映射 $\{f(x) \mid x \in a\} = \{f(a_1), \dots, f(a_n)\}$.



标准型 $\mathcal{D}$ -NF 与 $\mathcal{M}$ -NF

定义 $\mathcal{D}$ -NF, $\mathcal{M}$ -NF

我们定义表达式的标准型, 从而判断两个表达式是否一定相等。两个表达式一定相等意味着对任意满足我们上述描述的具体代数结构 $\mathcal{D}$, $\mathcal{M}$, 以及每个表达式中的变量取对应代数结构中的任意元素, 其计算出的表达式值都相等。

$\mathcal{D}$ -NF 和 $\mathcal{M}$ -NF 归纳定义如下:

对每个类型 $\mathcal{D}$ 的变量 x , $\mathcal{D}\text{-NF}(x) = \{(x)\}$, 即一个只含有 x 的列表的可重集合。

对每个类型 $\mathcal{M}$ 的变量 u , $\mathcal{M}\text{-NF}(u) = (u)$, 即一个只含有 u 的列表。

如果 a, b 都是 $\mathcal{D}\text{-expr}$, 则 $\mathcal{D}\text{-NF}(a + b) = \mathcal{D}\text{-NF}(a) + \mathcal{D}\text{-NF}(b)$, 这里 $+$ 表示两个可重集合的不去重并。

如果 a, b 都是 $\mathcal{M}\text{-expr}$, 则 $\mathcal{M}\text{-NF}(a * b) = \mathcal{M}\text{-NF}(a) + \mathcal{M}\text{-NF}(b)$, 这里 $+$ 表示列表连接。

如果 a 是一个 $\mathcal{D}\text{-expr}$, b 是一个 $\mathcal{M}\text{-expr}$, 则 $\mathcal{D}\text{-NF}(a * b) = \{x + \mathcal{M}\text{-NF}(b) \mid x \in \mathcal{D}\text{-NF}(a)\}$.



标准型 D-NF 与 M-NF

- 这里可以对应于我们很熟悉的线段树区间修改查询来理解。
- M-NF 的类型是**列表**，因为其维护的是一个区间上的标记，标记是半群元素，需要按顺序下放。列表形如 $(x, u_1, u_2, \dots, u_k)$ ，其中 u_i 为类型 M 的元素，表示依次进行了修改 $u_1, u_2, \dots, u_k$ 。
- D-NF 的类型是**列表构成的可重集合**，因为其维护的是区间的交换半群信息，每个点的信息是一个列表。可重集合中的每个列表形如 $(x, u_1, u_2, \dots, u_k)$ ，其中 x 为类型 D 的元素， u_i 为类型 M 的元素，表示初始点值为 x ，之后依次进行了修改 $u_1, u_2, \dots, u_k$ 。



标准型 D-NF 与 M-NF

- D-NF 的合并代表将两个区间的信息合并。
- M-NF 的合并代表将一个区间上的两个标记合并。
- D-NF 和 M-NF 的合并代表将这个 M-NF 对应的标记作用到 D-NF 中的每个元素上，这里需要对每个 D-NF 中的元素（类型为列表）与 M-NF（类型为列表）进行列表连接。
- 列表连接的细节：假设 D-NF 中的一个元素为 $(x, u_1, u_2, \dots, u_k)$ ，M-NF 为 $(u'_1, u'_2, \dots, u'_i)$ ，则连接后这个 D-NF 中的元素变为 $(x, u_1, u_2, \dots, u_k, u'_1, u'_2, \dots, u'_i)$ 。



引理 1

引理 1

对于具体的代数结构 $\mathcal{D}$ 和 $\mathcal{M}$ ，如果两个表达式有同一个标准型，则对任何变量的具体取值，这两个表达式都相等。

证明：

一个 $\mathcal{M}$ -NF 是形如 $(u_1, u_2, \dots, u_n)$ 的列表，每一个这样的标准型的 $\mathcal{M}$ -expr 都可以被简化为 $u_1 * u_2 * \dots * u_n$ ，通过一直使用 $\mathcal{M}$ 的结合律。

一个 $\mathcal{D}$ -NF 是形如 $\{(x_k, u_{k,1}, \dots, u_{k,n_k}) \mid k = 1, 2, \dots, N\}$ 的可重集合，每一个这样的标准型的 $\mathcal{D}$ -expr 都可以被简化为 $\sum_{k=1}^N x_k * (u_{k,1} * \dots * u_{k,n_k})$ ，通过一直使用 $\mathcal{D}$ 和 $\mathcal{M}$ 的结合律覆写，以及使用 $\mathcal{D}$ 的交换律和结合律将每一项交换为正确的顺序。

覆写的具体步骤在表达式的递归结构那一部分有描述。



引理 2

引理 2

如果两个表达式有不同的标准型，则一定存在两个具体的代数结构 $\mathcal{D}$, $\mathcal{M}$ 以及具体的变量取值，使得这两个表达式的值不同。

证明：

我们构造一个具体的代数结构，令 $\mathcal{M}$ 是包含了所有 由变量名构成的列表 的集合。

$\forall x, y \in \mathcal{M}$, $x * y$ 定义为列表连接 $x + y$.

令 $\mathcal{D}$ 是包含了所有 由变量名构成的非空列表 的可重集合 的集合。

$\forall x, y \in \mathcal{D}$, $x + y$ 定义为可重集合 x, y 的不去重并。

$\forall x \in \mathcal{D}$, $y \in \mathcal{M}$, $x * y$ 定义为 $\{z + y \mid z \in x\}$.

我们代入 $x := (x)$ 给每个 $\mathcal{M}$ 中的元素，以及 $x := \{(x)\}$ 给每个 $\mathcal{D}$ 中的元素，这样标准型会和表达式的值完全一样。这样两个表达式有同样的值当且仅当这两个表达式有同样的标准型。



中国计算机学会
China Computer Federation



隔离机制

- 让多个阶段之间互相不能复用中间结果。
- 每个阶段通过构造使得进行的操作为 k 次范围修改，一次全局查询。

我们希望证明，表达式中存在很多 $+$, $*$ 运算，每个运算的结果互不相同，因此运算结果无法复用，且每个操作的答案不是之前阶段的中间结果，因此每个阶段都至少需要这么多运算。

实际计算构成一个与初值和修改的具体取值无关的计算图，其中包含无入度的初值，2 入度的计算结果 $\mathcal{D} + \mathcal{D}, \mathcal{D} * \mathcal{M}, \mathcal{M} * \mathcal{M}$ ，其中某些结点对应某次查询的答案，

对每个对应查询答案的结点，我们都可以将其展开为表达式，根据其文法构成树，部分结点可能重复，但不重复的结点一定对应计算图上的不同结点。



证明

一个阶段的全局查询的表达式，至少需要包含这个阶段的所有修改操作，因为这个全局查询的表达式对应的标准型中出现了这些操作。同时不能包含之后阶段的修改操作，因为表达式中一旦出现某个变量，则在对应的标准型中也会出现；仅考虑这个阶段及以前的操作，得到的表示查询结果的表达式中没有出现后面的操作，但答案正确；如果实际计算对应的表达式中包含了后面的操作，且也保证答案正确，则违反引理 2。

对一个表达式 e ，仅考虑当前阶段的修改操作划分的等价类 $E(e)$ 。

等价类 $E(e)$ 定义为，标准型中，每个元组仅保留当前阶段操作的元素后，留下的部分构成的集合。

对应的范围 $I(e)$ 定义为涉及的初值构成的集合。

例如当前阶段包含 (m_4, m_5, m_6) ：

e 的标准型是一个由 **list** 构成的 **multiset**：

$\{(x_1, m_1, m_2, m_3, m_4), (x_2, m_3, m_4, m_5), (x_3, m_3, m_4, m_6), (x_4, m_4, m_5)\}$ 。

$E(e)$ 是一个由 **list** 构成的 **set**： $\{(m_4), (m_4, m_5), (m_4, m_6)\}$ 。

$I(e)$ 是一个由表示 $\mathcal{D}$ 上的元素的变量构成的 **set**： $\{x_1, x_2, x_3, x_4\}$ 。

我们对这—一个阶段中进行查询的那—个 $\mathcal{D}$ -**expr** 进行讨论：

第一种情况，若这个 $\mathcal{D}$ -**expr** 可以被表示为 $t_1 + t_2$ 的形式，其中 $t_1, t_2 \in \mathcal{D}$ ：

$$E(t_1 + t_2) = E(t_1) \cup E(t_2)$$

$$I(t_1 + t_2) = I(t_1) + I(t_2)$$



中国计算机学会
China Computer Federation



证明

如果 $E(t)$ 包含非空序列, 统计 $+$ 的次数, 递归到 t_1, t_2 。

第二种情况, 若这个 $\mathcal{D}$ -**expr** 可以被表示为 $t * m$ 的形式, 其中 $t \in \mathcal{D}, m \in \mathcal{M}$:

$$E(t * m) = \{x + m \mid x \in E(t)\}$$

$$I(t * m) = I(t)$$

如果 $|E(t)| = 1$, 结束;

否则 $E(t)$ 包含非空序列, 这说明 t 的值不可能在之前的阶段求得, 统计 $*$ 的次数, 递归到 t 。

在递归过程中, 每次 $+$ 操作对应了一次对 I, E 集合的分裂操作, 而边界情况是 $|E(t)| = 1$,

在 $\mathcal{D} + \mathcal{D}$ 操作中 I 分裂, 在 $\mathcal{D} * \mathcal{M}$ 操作中 I 不变, 这保证了每个 $\mathcal{D} + \mathcal{D}$ 、 $\mathcal{D} * \mathcal{M}$ 子问题对应的表达式互不相同。

定义 $E_0(e)$ 为 $E(e)$ 去除空序列后的 **set**。

在 $\mathcal{D} + \mathcal{D}$ 操作中,

$$E_0(t1 + t2) = E_0(t1) \cup E_0(t2)$$

$$1 \leq |E_0(t1 + t2)| \leq |E_0(t1)| + |E_0(t2)|$$

在 $\mathcal{D} * \mathcal{M}$ 操作中,

$$1 \leq |E_0(t * m)| \leq |E_0(t)| + 1$$

因此 $\mathcal{D} + \mathcal{D}$ 和 $\mathcal{D} * \mathcal{M}$ 次数之和 $\geq |E_0(e)| \geq |E(e)| - 1$ 。



证明

在上述过程中, 如果 $E_0(e)$ 非空, 我们就需要进行一些代数结构上的运算, 从而获得一些新的子表达式 $e_1, \dots, e_k$, $0 \leq k \leq 2$, 满足 $\sum_{i=1}^k |E_0(e_i)| \geq |E_0(e)|$. $I(e)$ 在第一种情况下会被分裂为两个不相交集合, 第二种情况下不变。所有在上述过程中产生的子表达式都两两不同。使用数学归纳法我们可以得出计算表达式 e 的值的代数结构上的运算次数在这个阶段中为 $|E_0(e)|$, 不比 $|E(e)| - 1$ 小。

我们选择 $G(n) + 1$, 满足 $F(G(n)) \leq n$, $F(G(n) + 1) > n$ 做为一个阶段中的修改查询次数。

如果 $m > G(n)$, 复杂度下界为 $\Omega(\frac{mn}{G(n)+1})$;

如果 $m \leq G(n)$, 一次没有任何修改操作的全局查询需要 $n - 1$ 次操作来合并所有的初始权值, 所以在这个情况下复杂度下界为 $\Omega(n + \frac{mn}{G(n)+1})$ 。

对于 $F(k) = \Theta(k^c)$, $c > 1$ 的情况, $G(k) = \Theta(k^{1/c})$, 我们有一个 $\Theta(n + mn^{1-1/c})$ 确界。

对于 $\forall c, F(k) = \omega(k^c)$ 的情况, 我们的算法也达到了复杂度确界。然而如果 $F(k) = \tilde{O}(k)$, 这个界不紧, 因为分治算法导致的额外开销更大。



中国计算机学会
China Computer Federation



具体范围的划分树

- 设计一种点集划分树，每次修改查询都只递归到少数子树中。
- 不同具体范围的划分树形态非常不同，目前对常见的几种范围有人专门设计了划分树。



中国计算机学会
China Computer Federation



序列区间

- 区间的最优点集划分树为线段树。
- 操作的时间复杂度为 $O(n+m\log n)$ 。
- $F(k)=\Theta(k)$, 代入我们的下界可以证明这个数据结构是最优的。



中国计算机学会
China Computer Federation



树上简单路径

- 区间的最优点集划分树为静态 LCT, top tree, 点分树 等。
- 操作的时间复杂度为 $O(n+m\log n)$ 。
- $F(k)=\Theta(k)$, 代入我们的下界可以证明这个数据结构是最优的。



中国计算机学会
China Computer Federation



二维平面矩形

- 二维平面矩形的最优划分树为 KDT。
- 操作的时间复杂度为 $O(n + m \sqrt{n})$ 。
- $F(k) = \Theta(k^2)$, 代入我们的下界可以证明这个数据结构是最优的。



中国计算机学会
China Computer Federation



高维正交范围

- 高维正交范围的最优划分树为 KDT。
- 对 d 维的情况，操作的时间复杂度为 $O(n + mn^{1-1/d})$ 。
- $F(k) = \Theta(k^d)$ ，代入我们的下界可以证明这个数据结构是最优的。



高维满点集正交范围

- 高维满点集正交范围的最优划分树为 KDT，四分树。
- 对 d 维的情况，操作的时间复杂度为 $O(n^d + mn^{(d-1)})$ 。
- $F(k) = \Theta(k^d)$ ，代入我们的下界可以证明这个数据结构是最优的。



中国计算机学会
China Computer Federation



二维半平面

- KDT 复杂度是错的。
- 圆之类的范围 KDT 也很好卡。

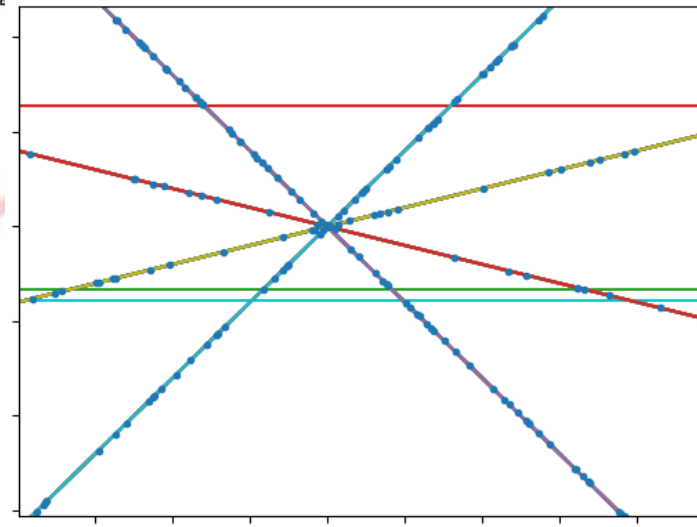
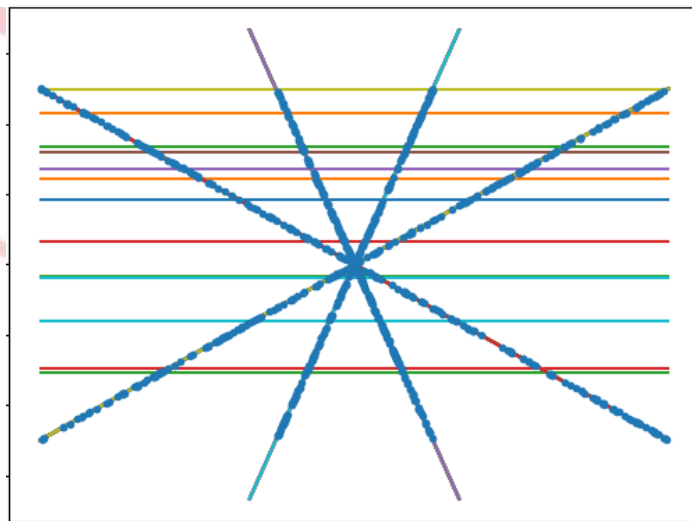
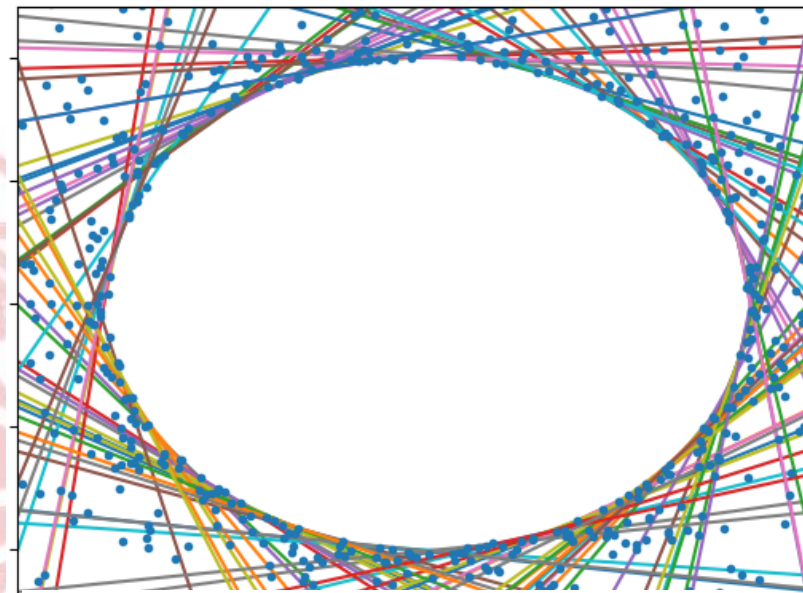
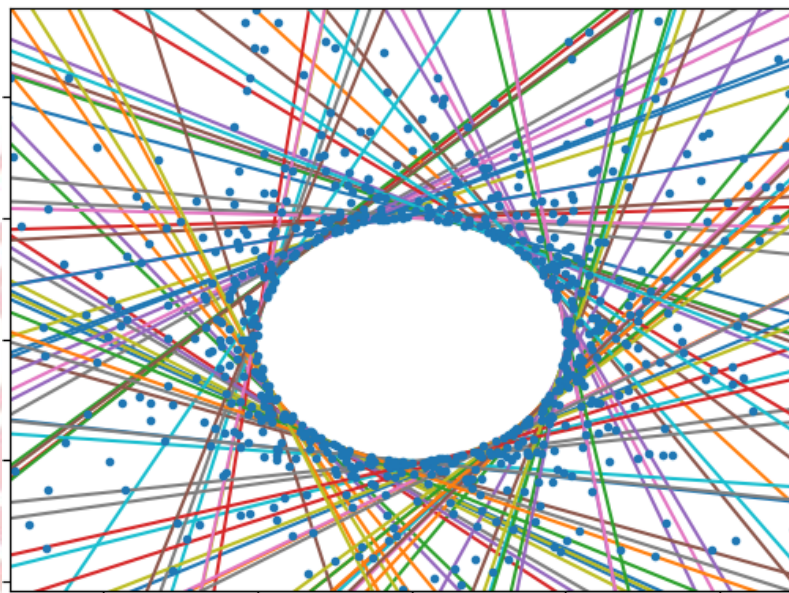
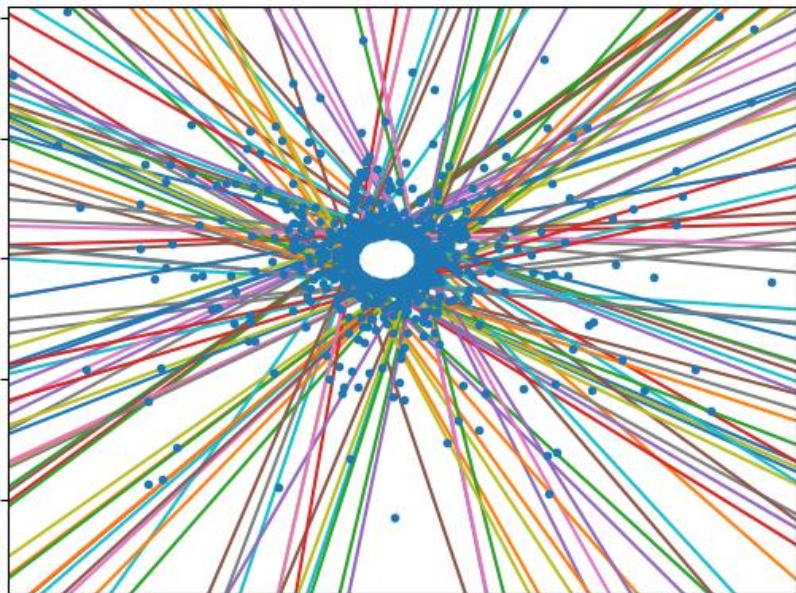




中国计算机学会
China Computer Federation



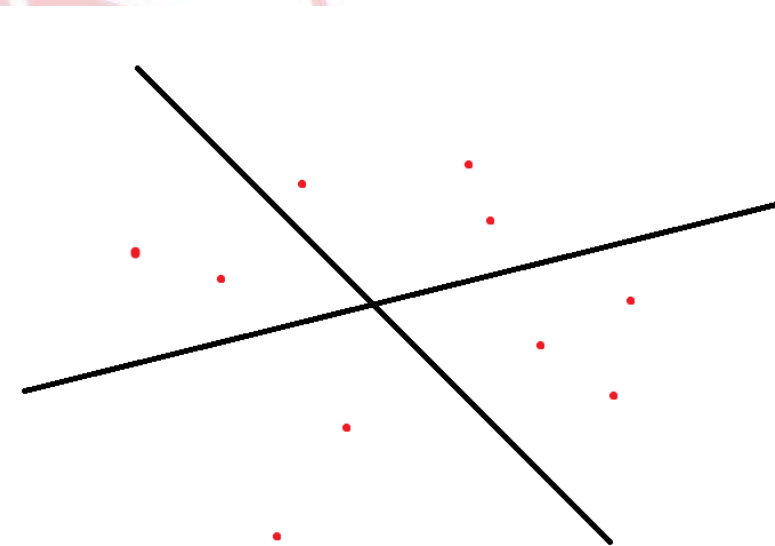
怎么卡KDT乱搞





二维半平面

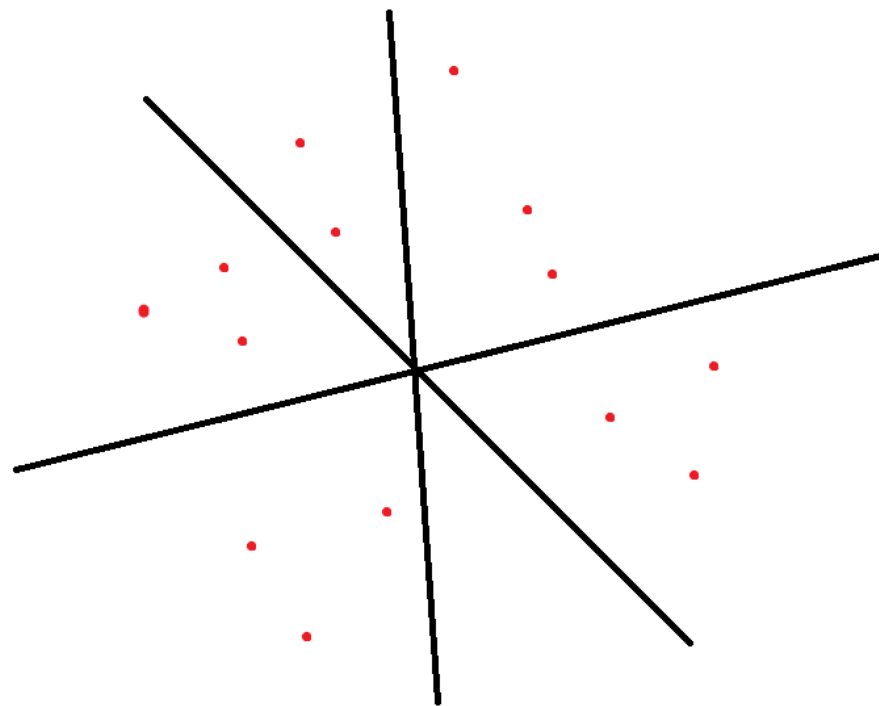
- 四分树是一种比暴力更优的半平面划分树。
- 每次通过两条斜线将平面点集均分为 4 份。
- 进行半平面修改查询时只会递归进 4 个儿子中的 3 个。
- $T(n)=3T(n/4)$, 解得 $T(n)=O(n^{0.79})$ 。
- 时间复杂度为 $O(n+mn^{0.79})$ 。





二维半平面

- 六分树是一种比四分树更优的半平面划分树。
- 每次通过三条斜线将平面点集均分为 6 份。
- 进行半平面修改查询时只会递归进 6 个儿子中的 4 个。
- $T(n)=4T(n/6)$, 解得 $T(n)=O(n^{0.77})$ 。
- 时间复杂度为 $O(n+mn^{0.77})$ 。
- 建树可以用一种类梯度下降的方法。





中国计算机学会
China Computer Federation



二维半平面

- Optimal Partition Tree 是最优的二维平面单纯型范围修改查询数据结构。
- 即存在一种划分点集的方法，使得任何二维单纯型可以用 $O(\sqrt{n})$ 个子树所表示的点集的和表示。
- 这个数据结构常数较大并且难以实现。
- $F(k) = \Theta(k^2)$ ，代入我们的下界可以证明这个数据结构是最优的。



中国计算机学会
China Computer Federation



高维单纯型

- Optimal Partition Tree 可以拓展到高维单纯型范围上，同时保证理论最优。
- 对 d 维的情况，操作的时间复杂度为 $O(n+mn^{1-1/d})$ 。
- $F(k)=\Theta(k^d)$ ，代入我们的下界可以证明这个数据结构是最优的。



圆范围

- 二维的圆范围可以看做是特殊的三维半空间范围。
- 设圆方程为 $(x-a)^2+(y-b)^2 \leq c$ ，可以将平面上的点 (d,e) 转换为三维点 (d,e,d^2+e^2) ，圆转换为半空间 $x^2-2ax+a^2+y^2-2by+b^2 \leq c$ ，即 $(-2a)x+(-2b)y+1(x^2+y^2) \leq -a^2-b^2+c$ ，即半空间一侧点。
- 这里的特殊条件是转换后空间中点全部在一个抛物面上，因为第三维的坐标为前两维的坐标的二次函数。

圆范围

- 我见过的已有的最优圆范围划分树为使用上述方法将圆转换为半空间后使用 Optimal Partition Tree。
- 这里和我们算法能达到的上界，以及能证明的下界不符。
- 对于平面上的 k 个圆，任意两个圆最多有 2 个交点，因为这里是平面图的结构，所以点数和面数同阶，所以 $F(k) = \Theta(k^2)$ 。



中国计算机学会
China Computer Federation



点集划分树的优势

- 在范围查询问题上，基于好的点集划分树的算法可以比我们的最优范围修改查询算法更优。
- 能在线。





中国计算机学会
China Computer Federation



半平面范围查询

- 没有修改操作，只有查询操作，查询为半平面的一侧的交换半群信息。





Solution1 离线最优范围修改查询

- 我们假装有修改操作，直接使用我们的修改查询算法。
- 代数结构上的运算次数 $O(n+mn^{0.5})$ 。
- 总时间复杂度 $O(n+mn^{0.5}\log n)$ 。
- 离线算法。



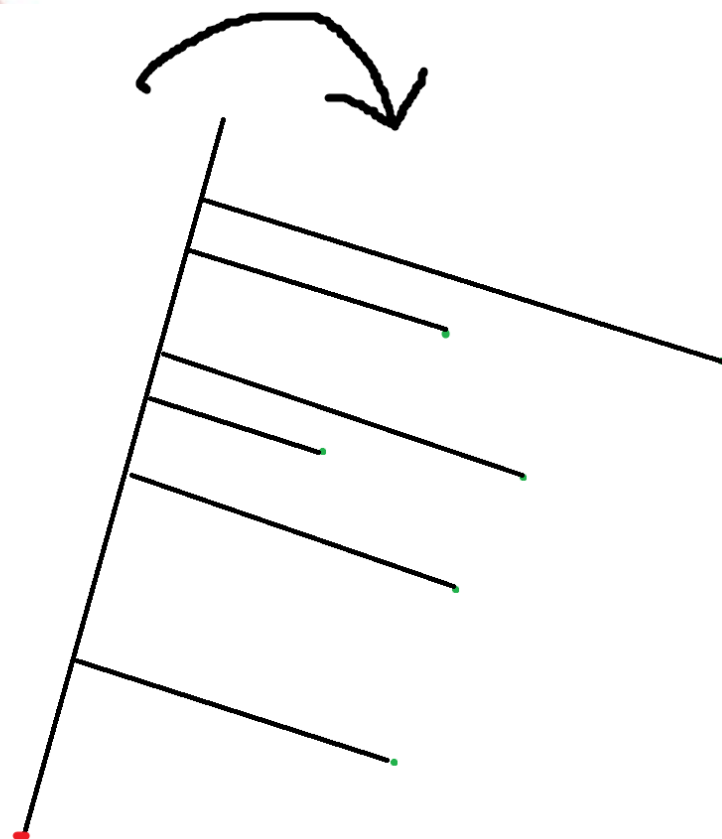
Solution2 旋转扫描线

- 将点集随意划分为大小为 B 的块。
- 因为两个形式不同的半平面可能实际上对应了同一个点集，这里定义半平面的标准型为将询问的半平面先一直向下平移，直到碰到第一个点，之后将询问的半平面一直顺时针方向旋转，直到碰到第一个点。
- 可以证明这样的变换是不会改变这个半平面一侧的点集的。
- 这样考虑 B 个点的情况下，只可能存在 $O(B^2)$ 个半平面的标准型。
- 在线算法。



Solution2 旋转扫描线

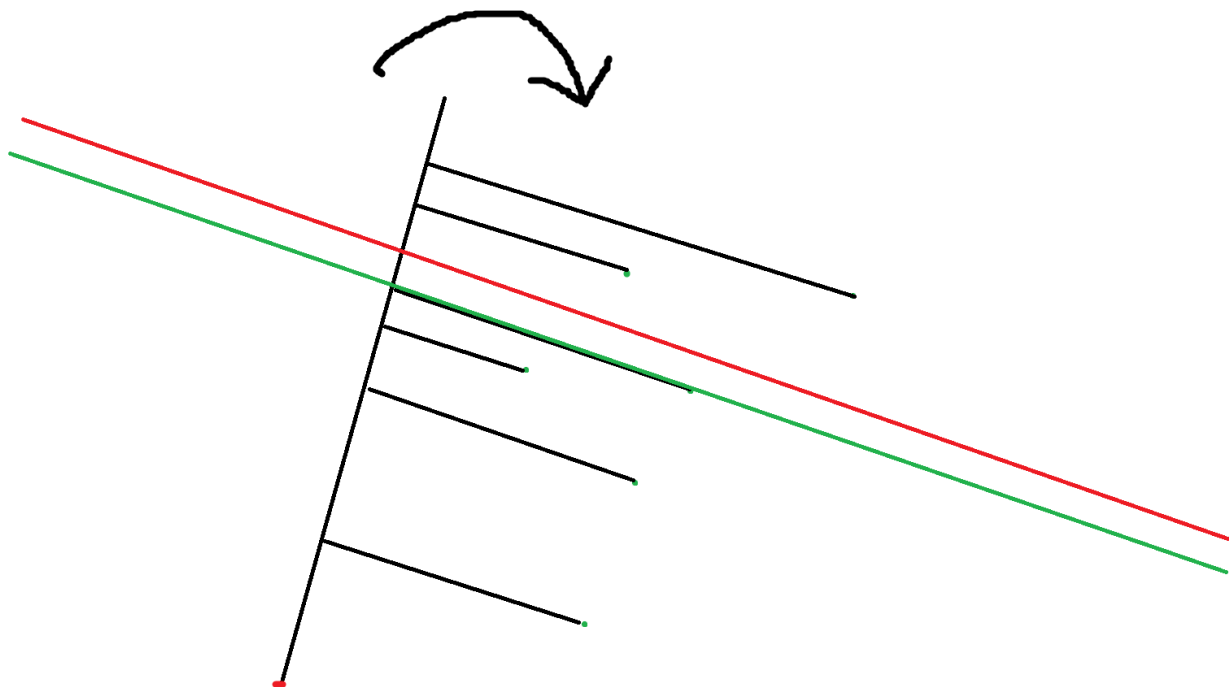
- 旋转扫描线的过程可以认为是我们将半平面的斜率从 $-\infty$ 到 $+\infty$ ，同时维护当前情况下所有不同的半平面标准型。
- 可以取任意一个点当原点，发出一条射线。
- 射线旋转的过程对应了旋转扫描线的过程。
- 在任意一种情况下，所有不同的半平面标准型
- 即所有点引向射线的垂线。
- 扫描线过程中可能交换相邻两个点的相对顺序
- 这样的交换只会发生 $O(B^2)$ 次。





Solution2 旋转扫描线

- 对于一次半平面范围询问，可以等扫描线扫到对应斜率时，在维护相对顺序的数据结构上求前驱后继（这里和平面图点定位的问题类似，必须二分，因为 key 是动态算的，不能使用高效的前驱数据结构）来得到一个被维护的范围做为答案。
- 动态维护当前旋转扫描线的序列
- 只需要支持前缀，后缀查询
- 交换相邻两点时可以 $O(1)$ 改。





中国计算机学会
China Computer Federation



Solution2 旋转扫描线

- 取 $B=m^{0.5}$ 。
- 代数结构上的运算次数 $O(nm^{0.5})$ 。
- 总时间复杂度 $O(n+nm^{0.5}\log n)$ 。



修改查询与只查询的区别

- 范围修改查询问题一般不会因为操作多而降低复杂度，如 $O(n+mn^{0.5})$ 。
- 范围查询问题一般会因为操作多而降低复杂度，如 $O(nm^{0.5})$ 。
- 可以考虑询问非常多的例子，比如 $m=\Theta(n^2)$ 的情况，此时可以暴力预处理出所有可能的询问，时间复杂度为 $O(n^2)$ 。



Solution3 旋转扫描线-plus

- 对于分子分母值域为 k 的分数，可以用 $\text{poly}(k)$ 精度的浮点数表示。
- 可以发现数据结构维护时每次交换是 $O(1)$ 的，复杂度带 $\log$ 的部分在于如何找出交换的时刻和位置。
- 可以做对两两点对连线的一次极角基数排序来预处理出所有交换的时刻和位置，这样有一部分是不带 $\log$ 的。
- 代数结构上的运算次数 $O(nm^{0.5})$ 。
- 总时间复杂度 $O(n+n(m\log n)^{0.5})$ 。
- 在线算法。



Solution4 Optimal Partition Tree

- 我们假装有修改操作，直接使用我们的最优修改查询点集划分树。
- 代数结构上的运算次数 $O(n+mn^{0.5})$ 。
- 总时间复杂度 $O(n+mn^{0.5}\log n)$ 。
- 在线算法。



Solution5 分治后分块

- 我们先使用最优点集划分树对点集进行分治，之后在较小的子树上使用旋转扫描线的算法来预处理出所有可能的情况。
- 点集划分树需要是重量平衡的。
- 我们对大小 $\Theta(m^{2/3}n^{1/3})$ 的子树，跑旋转扫描线算法预处理出 $O(m^{4/3}n^{2/3})$ 个半平面标准型。
- 当在点集划分树上递归时，递归到这一层就会停止递归。
- 代数结构上的运算次数 $O(n^{2/3}m^{2/3})$ 。
- 总时间复杂度 $O(n^{2/3}m^{2/3}\text{polylog}n)$ 。
- 在线算法。这里不保证内容完全正确，因为那篇论文我看不懂。



随机数据半平面范围查询

- 没有修改操作，只有查询操作，查询为半平面的一侧的交换半群信息。
- 因为随机数据会比较好写，所以保证所有点都在一个规定的矩形中两维独立地均匀随机生成，询问的随机方式可以有很多种不同的，一般来说性质差不多。



Solution1 旋转扫描线-plus-plus

- 当数据随机时可以使用插值查找来代替二分。
- 代数结构上的运算次数 $O(nm^{0.5})$ 。
- 总时间复杂度 $O(n+n(m\log\log n)^{0.5})$ 。
- 在线算法。
- 随机数据。



中国计算机学会
China Computer Federation



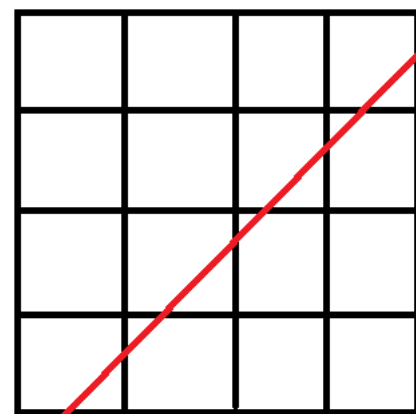
Solution2 KD树

- 当数据随机时，KD树是最优点集划分树。（未证明的结论，不过随机数据测下来看上去是的，所以我们就先假装是）
- 于是上述方法的最优点集划分树可以使用 KDT 来实现。
- 随机数据。



Solution3 平面分块

- 将平面划分为 $\text{sqrtn} \times \text{sqrtn}$ 个块。
- 因为点集均匀随机，所以一个块中期望只有常数个点。
- 这样块内询问的期望复杂度为 $O(1)$ 。
- 半平面可以看做是 $O(\text{sqrtn})$ 行的前缀信息查询， $O(\text{sqrtn})$ 次块内查询。
- 代数结构上的运算次数 $O(nm^{0.5})$ 。
- 总时间复杂度 $O(n + nm^{0.5})$ 。
- 这个是写出来最快的方法。
- 随机数据。





中国计算机学会
China Computer Federation



[CTS2022] 袜子

- 半平面小Z的袜子。

给定 n 个点 (x_i, y_i, c_i) , $i = 1, 2, \dots, n$, 共有 m 次查询操作, 每次查询给定 A, B, C , 问满足 $Ax_i + By_i + C < 0$, $Ax_j + By_j + C < 0$, $c_i = c_j$ 的二元组 (i, j) 的个数。





中国计算机学会
China Computer Federation

[CTS2022] 袜子



数据范围：

对 5% 的数据, $n, m \leq 10^3$;

对另外 10% 的数据, $c_i \leq 2$;

对另外 15% 的数据, $c_i \leq 100$;

对另外 15% 的数据, $\max(|x_i|, |y_i|) = 10^6$;

对另外 15% 的数据, $|A| = |B| = 1$;

对另外 10% 的数据, $n \leq 20000, m \leq 200000$;

对于其余数据, 无特殊约束。

每部分数据构成子任务, 无依赖关系。

所有数据满足：

$1 \leq n \leq 50000$;

$1 \leq m \leq 500000$;

$A^2 + B^2 > 0$;

$-10^9 \leq x_i, y_i, A, B, C \leq 10^9$;

$1 \leq c_i \leq n$;

所有数值为整数;

当 $i \neq j$ 时, $x_i \neq y_i$ 或 $x_j \neq y_j$ 。

对于除了子任务 4 以外的数据, 满足 n 个点的 x, y 坐标分别在某个预设的区间内均匀随机选取, 并保证没有重复的点, 且对于第 i 个点, c_i 和 x_i, y_i 是分别独立地随机选取的, 但 c_i 的分布没有特殊限制。



Solution1

题意是多次查询半平面内的点的每种点权 (c_i 是点权, (x_i, y_i) 是平面上的坐标) 出现次数的平方和。

将点按点权排序, 按大小 B 分块, 对每个询问查询出每块内的信息 (半平面内第一种点权和最后一种点权的出现次数, 以及中间的点权的出现次数平方和), 即可合并得到答案。

使用旋转扫描线 (维护点按 $x_i \cos \theta + y_i \sin \theta$ 排序得到的序列, θ 从 0 增大到 2π) 处理块内的询问, 问题转化为维护点构成的序列, $O(B^2)$ 次交换相邻位置, m 次查询前缀内的信息。使用链表维护每种点权的点在序列中的顺序, 每个点另外维护当前在这种点权的点中是从左到右第几个, 即可在交换时更新每个点对应的前缀信息。

时间复杂度 $T(n, m) = O\left(\left(\frac{n}{B} + 1\right)(B^2 + m) \log n\right) = O\left(n\sqrt{m} \log n\right)$, 这个 $\log n$ 可以通过一些方法去掉, 不过本题时限使用的是没有去掉 $\log n$ 的一份 std 的时限两倍。



Solution2

- 用一个 OI 常见套路方法，将点集和询问的范围反演，计算点集对询问的贡献。
- 点 (a,b) 在半平面 $cx+dy<e$ 的一侧，假设 e 非负，半平面变成 $(c/e)x+(d/e)y<1$ ，即转换为点 $(c/e,d/e)$ 在半平面 $ax+by<1$ 的一侧。
- 对每个颜色的点，算该颜色对每个询问的贡献。
- 可以做一种特殊形式的修改操作，即每个 KDT 结点维护一个额外的域 cnt ，枚举颜色 x ，对所有颜色 x 的点，将包含了该点的半平面的 $cnt++$ ，进行完了这个颜色的所有操作后，我们访问刚才访问到过的所有 KDT 结点，对这些 KDT 结点进行子树答案加。



ICPC2022 沈阳H 赫露艾斯塔

• 半平面莫队。

给定 n 个互不相同的点 (x_i, y_i) , m 个集合 $S_j = \{(x_i, y_i) \mid A_j x_i + B_j y_i + C_j > 0\}$, 你需要找出一个 $1, 2, \dots, m$ 的排列 $p_1, \dots, p_n$, 使得 $|S_{p_1}| + \sum_{i=2}^m |S_{p_i} \oplus S_{p_{i-1}}| \leq M$, M 是给定的常数, $A \oplus B$ 表示 $(A \cup B) - (A \cap B)$ 。

输入格式

第一行两个整数 n, m ;

接下来 n 行每行两个整数表示 x_i, y_i ;

接下来 m 行每行三个整数表示 A_j, B_j, C_j 。

输出格式

输出 m 行, 依次表示 $p_1, \dots, p_n$ 。

数据范围

$$1 \leq n \leq 10^5$$

$$1 \leq m \leq 2 \times 10^5$$

$$M = 1.8 \times 10^8$$



Solution

期望最优半平面莫队。

问题转化为维护点集，初始为空，支持加点、删点不超过 M 次，使得每个半平面对应的点集在这个过程中都出现过至少一次。

随机选取 B 个点，每个点进行以这个点为中心的旋转扫描线（代价 $B \cdot 2n$ ），每个半平面需要期望 $2\frac{n}{B}$ 代价移动到半平面内最接近分界直线的点的旋转扫描线过程的对应位置。 B 个旋转扫描线都从同一个斜率开始和结束，则把它们连起来的代价不超过 n 。

期望总代价 $n + 2nB + 2\frac{mn}{B} = 4n\sqrt{m}$ ，如果觉得方差大可以多随机几次。

朴素实现的时间复杂度 $O(mB) = O(m^{1.5})$ ，可以通过此题。

可以优化到 $O((B^2 + m) \log B) = O(m \log m)$ 。



P7723 [Ynoi2007] TB5x

这是一道交互题。

二维平面上初始有 n 个点 (i, p_i) ，每个点有权值 d_i ， $0 \leq i < n$ ；

共 m 次操作，操作编号为 0 到 $m - 1$ ，按编号升序执行；

编号为 w 的操作给出 x_1, x_2, y_1, y_2 ，满足 $0 < x_1 < x_2 < n$ ， $0 < y_1 < y_2 < n$ 。

一个点 (x, y) 的网格坐标被定义为 $((x \geq x_1) + (x \geq x_2), (y \geq y_1) + (y \geq y_2))$ 。

依次进行以下步骤：

1. 查询网格坐标为 (X, Y) 的点的权值之和，记录在 $ans[X][Y]$ 。
2. 对每个网格坐标为 (X, Y) 的点，进行修改 $o[X][Y]$ 。
3. 所有点的坐标同时发生改变，对于一个网格坐标为 (X, Y) 的点，它的坐标从 (x, y) 变为 $(x + dx[X], y + dy[Y])$ ；
其中
 $dx[0] = 0, dx[1] = n - x_2, dx[2] = x_1 - x_2$ 。
 $dy[0] = 0, dy[1] = n - y_2, dy[2] = y_1 - y_2$ 。
 $x_1, x_2, y_1, y_2, o, ans$ 都是数组，下标对应操作的编号。

其中 d_i 属于抽象的数据类型 D ， $o[X][Y]$ 属于抽象的数据类型 O ；

D 上定义了抽象的运算 $+$ ： $D \times D \rightarrow D$ ；

D, O 上定义了抽象的运算 $\cdot$ ： $O \times D \rightarrow D$ ；

O 上定义了抽象的运算 $\cdot$ ： $O \times O \rightarrow O$ ；

ϵ_D 是 D 中的一个特殊的元素，称为单位元；

ϵ_O 是 O 中的一个特殊的元素，称为单位元；

这些操作满足性质：

对任意 $a, b, c \in D$ ，有 $a + b = b + a$ ， $(a + b) + c = a + (b + c)$ ， $a + \epsilon_D = \epsilon_D + a = a$ ；

对任意 $u, v, w \in O$ ，有 $(u \cdot v) \cdot w = u \cdot (v \cdot w)$ ， $u \cdot \epsilon_O = \epsilon_O \cdot u = u$ ；

对任意 $u, v \in O$ ， $a, b \in D$ ，有 $(u \cdot v) \cdot a = u \cdot (v \cdot a)$ ， $u \cdot (a + b) = (u \cdot a) + (u \cdot b)$ ， $\epsilon_O \cdot a = a$ ， $u \cdot \epsilon_D = \epsilon_D$ ；

执行每次 $+$ 或 $\cdot$ 运算有一定的代价，具体地，在计算 $a + b$ 或 $a \cdot b$ 时如果 a, b 都不是 ϵ_D 或 ϵ_O ，则代价为 1 ，否则代价为 0 。你需要保证每个答案正确，且总代价不能超过当前子任务的**代价上限**。

输入格式

下发的交互库以如下格式读取输入数据：

- 第一行： n
- 接下来 n 行： $p_i d_i$ (d_i 由两个整数表示)
- 第 $n + 2$ 行： m
- 接下来 m 行： $x1_i x2_i y1_i y2_i o_i$ (o_i 由 $9 * 4$ 个整数表示)

D 中的元素是 2×1 的矩阵， O 中元素是 2×2 的矩阵，矩阵中的元素是对 2^{32} 取模的整数；

$+$ 对应矩阵加法， $\cdot$ 对应矩阵乘法，具体可以参考下发的交互库的实现；

实际评测环境中输入输出格式以及 D, O 等可能有不同的定义；

输出格式

下发的交互库以如下格式打印你的答案：

- 对每个询问，输出 13 行，其中第 $1, 2, 3, 5, 6, 7, 9, 10, 11$ 行有两个整数，依次表示这次询问对应的 $ans[0][0], ans[0][1], ans[0][2], ans[1][0], ans[1][1], ans[1][2], ans[2][0], ans[2][1], ans[2][2]$ ，其余为空行；
- 向 `stderr` 打印总代价，以及在总代价超过代价上限时进行提示。





中国计算机学会
China Computer Federation



P7723 [Ynoi2007] TB5x

实现细节

你必须引用 `data.h` 头文件。

头文件中定义了数据类型 `Data` (D) 和 `Operation` (O)，你可以使用以下已定义的成员函数对类型为 `Data` 和 `Operation` 的数据进行操作：

```
void Data::add_eq(const Data &a)
```

`w.add(a)` 计算 $w + a$ ，并将结果保存在 w ，每次调用的代价在 w, a 都不是单位元时为1，否则为0；

```
void Data::add(const Data &a, const Data &b)
```

`w.add(a, b)` 计算 $a + b$ ，并将结果保存在 w ，每次调用的代价在 a, b 都不是单位元时为1，否则为0；

```
void Data::clr()
```

`w.clr()` 可以将 ϵ_D 保存在 w ，每次调用的代价为0；

```
bool Data::empty()const
```

`w.empty()` 判断 w 是否为 ϵ_D ，若是则返回 `true`，否则返回 `false`，每次调用的代价为0；

```
void Operation::apply(Data &a)const
```

`w.apply(a)` 计算 $w \cdot a$ ，并将结果保存在 a ，每次调用的代价在 w, a 都不是单位元时为1，否则为0；

```
void Operation::apply(Operation &u)const
```

`w.apply(u)` 计算 $w \cdot u$ ，并将结果保存在 u ，每次调用的代价在 w, u 都不是单位元时为1，否则为0；

```
void Operation::clr()
```

`w.print()` 可以将 ϵ_O 存储在 w ，每次调用的代价为0；

```
bool Operation::empty()const
```

`w.empty()` 判断 w 是否为 ϵ_O ，若是则返回 `true`，否则返回 `false`，每次调用的代价为0；

另外，你还可以使用 `Data` 或 `Operation` 类型的赋值运算符、拷贝构造函数或无参构造函数，以 `Data` 类型为例：

`w=u` 可以将 u 复制一份存储在 w ，每次调用的代价为0；

`Data w(u)`；或 `Data w=u`；可以将 u 复制一份存储在新定义的 w ，每次调用的代价为0；

`Data w`；可以将 ϵ_D 存储在新定义的 w ，代价为0；

`Operation w`；可以将 ϵ_O 存储在新定义的 w ，代价为0；

除了以上描述的对 `Data` 或 `Operation` 的操作外，其余操作根据情况可能被视为攻击评测系统。

`sizeof(Data)` 和 `sizeof(Operation)` 都不超过 64，此外交互库还需要不超过 64MB 的空间。时间和空间限制包括交互库使用的时间和空间。仅使用赋值、构造函数、`apply` 和 `add` 就可以写出正确的程序，其它函数可能可以提供便利。

你需要实现以下函数：

```
void solve(  
    const int n,  
    const int m,  
    const int p[],  
    const Data d[],  
    const int x1[],  
    const int x2[],  
    const int y1[],  
    const int y2[],  
    const Operation o[][3][3],  
    Data ans[][3][3])
```

- n ：点的个数；
- m ：操作的个数；
- p ：($i, p[i]$) 表示每个点的坐标， $0 \leq i < n$ ；
- d ： $d[i]$ 表示每个点的初始权值， $0 \leq i < n$ ；
- $x1, x2, y1, y2, o, ans$ ： $x1[i], y1[i], x2[i], y2[i], o[i]$ 表示每次操作的输入，你需要将相应的答案存储到 $ans[i]$ ， $0 \leq i < m$ 。



中国计算机学会
China Computer Federation

Solution





中国计算机学会
China Computer Federation



特殊代数结构的范围修改查询

- 当代数结构有比“双半群”模型更多的性质时，我们的算法下界不能用于说明问题的复杂度下界，于是可能利用代数结构的特殊性质降低问题的时间复杂度。





中国计算机学会
China Computer Federation



经典问题

- 维护一个 $n \times n$ 的二维平面，每个位置初始为 0
- 有 m 次操作：
 1. 矩形对 x 取 $\max$
 2. 查矩形 $\max$
- 强制在线



Solution

- 考虑使用树套树维护，第一维分治 x ，第二维分治 y 。
- 维护两个内层树，第一个表示所有子树内的标记的影响，第二个表示所有这个外层树节点上的标记的影响。
- 矩形修改的时候我们直接标记永久化，考虑第一维修改的 $O(\log n)$ 个线段树节点的虚树，我们在这 $O(\log n)$ 个外层树节点的内层树上进行区间 $\max$ 操作。
- 询问的时候需要查询 $O(\log n)$ 个外层树节点的内层树区间 $\max$ 。



Solution

- 询问的时候再把这这些外层树节点的虚树上每个查一下对应内层树区间的 max。
- 这样我们维护了子树内的贡献，维护了子树上层的贡献，所以正确性有保证。
- 矩形加矩形和也能这么做，不过有简单些的做法。
- 总时间复杂度 $O(n \log n + m \log^2 n)$ 。



P7711 [Ynoi2077] 3dmq

给定一个三维空间上 n 个点，每个点有坐标 X, Y, Z ，权值 a, b ， b 初始为 0。

有 m 个操作：

$x\ y\ z\ w$ ：求所有 $X_i \leq x, Y_i \leq y, Z_i \leq z$ 的 i 的 b_i 的和，求和结束后，将所有 $X_i \leq x, Y_i \leq y, Z_i \leq z$ 的点 i ，其 b 权值加上 $a_i \times w$ 。

对于 100% 的数据，满足 $1 \leq n, m \leq 5 \times 10^5$ ，初始点集以及权值为均匀随机生成， X, Y, Z 为 1 到 n 的排列， $0 \leq a, w \leq n$ ，操作为均匀随机生成，每次操作有 0.8 的概率其 w 为 0。

本题按照你的程序正确回答的询问个数给分，若你的程序正确回答了前 x 个询问，若 $2x \leq m$ ，你将得到 $\lfloor \frac{100x}{m} \rfloor \%$ 的分数，否则你将得到 $\lfloor 50 + 50 \times (\frac{2x-m}{m})^2 \rfloor \%$ 的分数。spj 将会在读取到第一个错误的答案或读取到输出末尾或读取完前 m 行时终止读取，之后的信息将被忽略，请勿在行末添加空格。

- 60s



中国计算机学会
China Computer Federation

Solution





P7899 [Ynoi2006] rprmq2

给定正整数 m ，你需要维护一个 $m \times m$ 的矩阵 $a_{i,j}$ ， $1 \leq i, j \leq m$ ，初值为 0。

共 m 次操作，每次操作给出 $x, y, z_{0,0}, z_{0,1}, z_{1,0}, z_{1,1}$ ，

首先对 $0 \leq p, q \leq 1$ 询问矩阵中满足 $[i \geq x] = p, [j \geq y] = q$ 的位置的值 $a_{i,j}$ 的最大值，答案记为 $w_{p,q}$ ，

然后对矩阵每个位置 (i, j) ，将其值 $a_{i,j}$ 加上 $z_{[i \geq x], [j \geq y]}$ 。

记号 $[c]$ 在条件 c 为真时值为 1，否则值为 0。

对于 100% 的数据， $2 \leq x, y \leq m \leq 2 \times 10^5$ ， $1 \leq z_{i,j} \leq 10^9$ ，所有数值为整数。

时限约为 20s，本题随机数据强度很高，只需要少数数据。

- 20s



中国计算机学会
China Computer Federation

Solution





中国计算机学会
China Computer Federation

谢谢大家

